Applied AI Engineer
Study Guide

APIs, FastAPI, SQL, Vector Databases, RAG, LLMs and System Design,
learned by building a RAG-based AI Fitness Coach

Built for: landing a Tier-1 Applied AI / Forward Deployed AI Engineering internship.

How to read the tags: sections marked CORE are things you must be able to recall instantly in an interview. Everything else is "know it exists and where to look it up".

Version 1 - October 2026

Contents

1. How to learn this permanently
2. The big picture: training vs RAG, and our roadmap
3. HTTP fundamentals
4. REST API design
5. FastAPI
6. Authentication, authorization and API security
7. Consuming external APIs (the core FDE skill)
8. Python essentials for backend work
9. SQL and relational databases
10. Embeddings and vector databases
11. The RAG pipeline and how to evaluate it
12. LLMs and LLM APIs
13. System design for AI applications
14. What makes a great Forward Deployed Engineer
15. Interview question bank with answers
16. Checklists and debugging toolkit

1. How to learn this permanently

Forgetting is normal: you lose most new information within a day or two unless you actively bring it back. Rereading this guide will *feel* productive but barely helps. These techniques actually work, ordered by impact.

The five techniques

1. **Retrieval practice.** Close the guide and write or say everything you remember about a topic, then check what you missed. Recalling strengthens memory; rereading only creates familiarity.
2. **Spaced repetition.** Review at growing intervals: 1 day, 3 days, 1 week, 2 weeks, 1 month. Use Anki (free) and write the cards yourself, in your own words, as questions.
3. **Build it.** You remember what you *did*. Tie every concept to a moment in the project. You type the code; the mentor reviews it.
4. **Feynman technique.** Explain a concept in plain language (Turkish or English) to a non-programmer. Where you stall or use jargon, you have a gap.
5. **Interleaving.** Mix topics when practicing (one HTTP question, then a RAG one, then SQL). Interviews ask in random order.

A realistic routine

When	What	Time
Daily	Anki reviews	10-15 min
Each build session	Write one step yourself, get it reviewed, add 3-5 sentences to `LEARNING_LOG.md`	45-90 min
End of each step	Answer that topic's questions from chapter 15 out loud	5 min
Weekly	Draw the whole system from memory and explain it	15 min

> **Good flashcard example.** Front: "401 vs 403?" Back: "401 = not authenticated (who are you?). 403 = authenticated but not allowed." Small, one idea, phrased as a question.

2. The big picture

Training a model vs providing context with RAG CORE

	Training / fine-tuning	RAG (Retrieval-Augmented Generation)
What changes	The model's weights	Nothing in the model; the prompt gets extra context at request time
Good for	Behavior, style, format, domain jargon	Facts, changing data, per-user data, citations
Updating knowledge	Retrain (hours, money)	Re-index a document (seconds)
Auditability	Hard: "where did it learn that?"	Easy: you can show the retrieved sources
Analogy	Memorizing the textbook	Open-book exam

Why RAG fits a fitness coach: biometrics change daily, advice must be grounded in vetted protocols (safety), knowledge updates should be cheap, and you cannot fine-tune a model per user per day. Fine-tuning may later adjust *tone*, never *facts*.

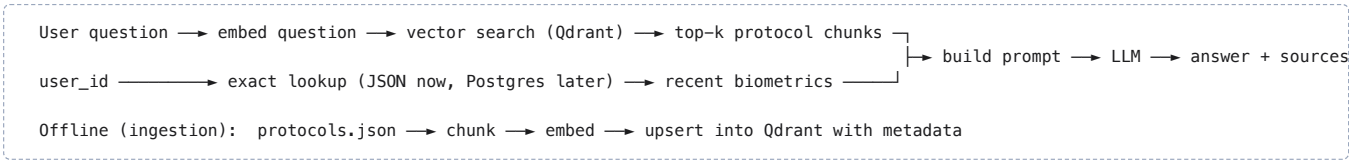
Also know: the cheapest option is plain prompting. The usual decision order is: prompt engineering, then RAG, then fine-tuning, then training from scratch (almost never).

Two kinds of context CORE

- **Structured** (HRV, sleep score, calories, weight): lives in SQL, fetched with an *exact* query ("user u1, last 7 days").
- **Unstructured** (recovery protocols, nutrition guidelines): lives in a vector DB, fetched by *semantic similarity* ("things related to CNS fatigue").

Putting numeric user data into a vector DB is a classic junior mistake: similarity search is fuzzy, and you need exact numbers.

The request flow



Project roadmap

Phase	What we build	Main lessons
1	Core RAG loop: FastAPI, mock JSON, OpenAI, embedded Qdrant	HTTP/FastAPI, embeddings, cosine similarity, vector search, prompt building
2	Retrieval quality and evaluation	Chunking, metadata filters, hybrid search, reranking, recall@k, MRR, LLM-as-judge
3	Persistence: Postgres, SQLAlchemy, Alembic, Docker	SQL, data modeling, migrations, dependency injection, rolling baselines
4	Google Health and FatSecret integrations	OAuth2, token refresh, pagination, retries, background jobs, idempotent upserts
5	Personalization and agents	Tool calling, structured outputs, guardrails, memory
6	Production readiness	JWT auth, async, streaming, caching, observability, CI, deployment

Phase 1 steps

1. Archive the generated scaffold, make the first commit *(done)*
2. Minimal FastAPI app with `/health` and Pydantic models *(done)*
3. Mock data: `data/protocols.json` and `data/users.json`
4. Embeddings + cosine similarity by hand with NumPy
5. Ingest into embedded Qdrant and search with filters
6. Prompt assembly and generation with `gpt-4o-mini`
7. Wire `POST /ask` end to end, returning answer + sources + scores

3. HTTP fundamentals

Every REST API is HTTP underneath. Understand HTTP and every framework becomes easy.

Anatomy of a request

```
POST /api/v1/ask HTTP/1.1                    <- method, path, version
Host: api.fitcoach.com                       <- headers (metadata)
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json

{"user_id": "u1", "question": "My HRV dropped, adjust leg day?"}   <- body
```

The response has the same shape: a status line (`HTTP/1.1 200 OK`), headers, and a body.

Methods, safety and idempotency CORE

Method	Purpose	Safe?	Idempotent?
`GET`	Read	Yes	Yes
`POST`	Create or trigger an action	No	No
`PUT`	Replace the whole resource	No	Yes
`PATCH`	Partial update	No	Not guaranteed
`DELETE`	Remove	No	Yes

- **Safe**: does not change server state.
- **Idempotent**: doing it once or ten times has the same effect. Networks fail and clients retry, so idempotent operations are safe to retry. `POST /payments` twice charges twice; that is why idempotency keys exist (chapter 7).

Status codes CORE

Code	Meaning	Typical use
200	OK	Successful read or update
201	Created	POST created a resource
202	Accepted	Work queued, finishes later (long AI jobs)
204	No Content	Successful DELETE
400	Bad Request	Malformed request
401	Unauthorized	Not authenticated: missing or invalid token
403	Forbidden	Authenticated, but not allowed
404	Not Found	Resource does not exist
409	Conflict	Duplicate or state conflict
422	Unprocessable Entity	Validation failed (FastAPI default)
429	Too Many Requests	Rate limit hit
500	Internal Server Error	Your bug
502 / 503 / 504	Bad Gateway / Unavailable / Gateway Timeout	An upstream service failed or is overloaded

Rule of thumb: **4xx = client's fault, 5xx = server's fault.** Only 429 and 5xx are generally worth retrying.

Headers worth knowing

- `Content-Type`: format of the body I am sending (`application/json`).
- `Accept`: format I want back.
- `Authorization`: credentials, usually `Bearer <token>`.
- `Retry-After`: how long to wait before retrying (with 429 or 503).
- `Cache-Control`, `ETag`: caching rules and resource versions.
- `X-Request-ID`: correlation ID to trace one request through all logs.

Statelessness and HTTPS

Stateless: each request carries everything the server needs (including auth). The server keeps no memory of the client between requests, so any of N identical servers can handle any request. This is what makes horizontal scaling easy.

HTTPS = HTTP over TLS: encrypts traffic and proves the server's identity. Always required in production, especially for health data.

4. REST API design

REST models your system as **resources** (nouns) acted on by HTTP **methods** (verbs).

URL design CORE

```
Good                                  Bad
GET    /users/u1                      GET  /getUser?id=u1
GET    /users/u1/biometrics?days=7    POST /fetchBiometricsForUser
POST   /users/u1/workouts             POST /createWorkout
PATCH  /users/u1/workouts/42          POST /updateWorkout
DELETE /users/u1/workouts/42          GET  /deleteWorkout?id=42
```

- Plural nouns, no verbs in paths. Nesting expresses ownership.
- Query parameters for filtering, sorting and pagination: `?days=7&sort=-recorded_on`.
- Pragmatic exception: non-CRUD actions are fine as POST, e.g. `POST /ask` or `POST /plans:generate`. Be consistent.

Versioning

Prefix with `/api/v1`. Once customers integrate you cannot break them. Additive changes (a new optional field) are fine in v1; breaking changes (renaming or removing a field, changing a type) go in v2.

Pagination

Style	Example	Pros / cons
Offset	`?limit=50&offset=100`	Simple; slow on big tables; items shift if data changes while paging
Cursor	`?limit=50&cursor=abc123`, response has `next_cursor`	Stable and fast; cannot jump to page 37. Google APIs use this (`pageToken`)

Never return unbounded lists.

Consistent error format

```
{"error": {"code": "user_not_found", "message": "User u9 does not exist", "request_id": "req_8f2a"}}
```

A machine-readable `code`, a human-readable `message`, and a `request_id` for support.

REST vs alternatives

Style	Use it when
REST	Default for public and customer-facing APIs; simple, cacheable, universal
GraphQL	Complex frontends that need flexible field selection; harder to cache and secure
gRPC	Internal service-to-service calls; binary, fast, strict protobuf schemas
WebSockets	Two-way real-time communication (live chat, multiplayer)
SSE (Server-Sent Events)	One-way server-to-client streaming; how LLM token streaming usually works

5. FastAPI

The stack

- **FastAPI**: the framework; maps paths to Python functions.
- **Uvicorn**: the server; handles networking and calls FastAPI.
- **ASGI**: the async interface between them (successor of WSGI used by Flask/Django).
- **Pydantic**: data validation from type hints.

Run with `uvicorn app.main:app --reload` ("in `app/main.py`, serve the object `app`; restart on file changes"). Interactive docs at `/docs` are generated from your type hints (OpenAPI / Swagger).

Why FastAPI over Flask? Type-driven validation, automatic docs, native async, high performance; the de-facto choice for ML serving.

Where inputs come from CORE

```
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/users/{user_id}/biometrics")
def list_biometrics(
    user_id: str,                          # in the path -> path parameter
    days: int = 7,                         # simple type -> query: ?days=14
    limit: int = Query(50, ge=1, le=200),  # query with validation
): ...

@app.post("/users/{user_id}/workouts", status_code=201)
def create_workout(user_id: str, workout: WorkoutCreate): ...   # Pydantic model -> JSON body
```

Pydantic and "validate at the boundary" CORE

```
from pydantic import BaseModel, Field

class AskRequest(BaseModel):
    user_id: str
    question: str = Field(..., min_length=3, max_length=2000)   # ... = required
```

FastAPI parses the JSON, validates it, and either gives you a typed object or returns **422** with a precise error before your code runs. Bad data is stopped at the door, so the rest of the system can trust its inputs.

`str | None = None` means the field is optional and defaults to `None`.

Separate input and output models

```
class WorkoutCreate(BaseModel):     # what the client sends
    name: str
    sets: int = Field(gt=0, le=20)

class WorkoutRead(WorkoutCreate):   # what the API returns
    id: int
    created_at: datetime

@app.post("/workouts", response_model=WorkoutRead, status_code=201)
def create(w: WorkoutCreate): ...
```

Clients must not set `id`; you must never return `password_hash`. `response_model` enforces the output contract and filters extra fields.

Errors

```
from fastapi import HTTPException

if user is None:
    raise HTTPException(status_code=404, detail=f"User {user_id} not found")
```

Dependency injection with `Depends` CORE

```
from fastapi import Depends, Header, HTTPException

def get_current_user(authorization: str = Header(...)) -> User:
    user = verify_token(authorization.removeprefix("Bearer "))
    if user is None:
        raise HTTPException(status_code=401, detail="Invalid token")
    return user

@app.get("/me/biometrics")
def my_biometrics(user: User = Depends(get_current_user)):
    return load_biometrics(user.id)
```

A dependency runs before the handler and injects its result. Use it for DB sessions, the current user, settings and clients. Two big wins:

1. No duplicated logic (auth, DB setup) across endpoints.
2. **Testability:** `app.dependency_overrides[get_llm] = lambda: FakeLLM()` lets you test without calling OpenAI.

Project structure: routers and layers

```
# app/routers/workouts.py
router = APIRouter(prefix="/api/v1/workouts", tags=["workouts"])

# app/main.py
app.include_router(workouts.router)
```

```
Router (HTTP only)  ⟶  Service (business logic)  ⟶  Data access (DB, Qdrant, external APIs)
```

Routers parse requests and shape responses; services decide what to do; the data layer talks to storage. Logic becomes reusable from scripts and jobs, and each layer is testable alone.

Lifespan: create expensive things once

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.qdrant = QdrantClient(path="./qdrant_data")   # startup
    yield
    app.state.qdrant.close()                                 # shutdown

app = FastAPI(lifespan=lifespan)
```

Middleware and CORS

Middleware wraps every request: logging, request IDs, timing, CORS. **CORS** is the browser rule that blocks a page on `app.com` from calling `api.com` unless the API allows that origin. "Works in Postman, fails in the browser" almost always means CORS. It is a browser protection, not a server security feature.

Sync vs async CORE

- `def` handlers run in a thread pool: blocking code is acceptable.
- `async def` handlers run on the event loop: everything inside must be non-blocking (`await httpx.AsyncClient().get(...)`, `AsyncOpenAI`).
- A blocking call (`time.sleep`, `requests.get`, sync DB driver) inside `async def` freezes all concurrent requests. Most common FastAPI bug.
- LLM apps are **I/O-bound** (waiting on OpenAI, DBs), which is exactly where async shines. CPU-bound work (embedding locally, heavy NumPy) belongs in worker processes.

Streaming LLM output

```
from fastapi.responses import StreamingResponse

@app.post("/ask/stream")
def ask_stream(payload: AskRequest):
    def tokens():
        stream = client.chat.completions.create(
            model="gpt-4o-mini", messages=build_messages(payload), stream=True
        )
        for chunk in stream:
            yield chunk.choices[0].delta.content or ""
    return StreamingResponse(tokens(), media_type="text/plain")
```

Users see text after a few hundred milliseconds instead of waiting for the full answer. Key metric: **time to first token (TTFT)**.

Long-running jobs

```
POST /ingest-jobs       -> 202 Accepted  {"job_id": "j1"}
GET  /ingest-jobs/j1    -> {"status": "running", "progress": 0.4}
```

Don't hold an HTTP request open for minutes. Queue the work (Celery, RQ, Arq; or `BackgroundTasks` for small jobs) and let the client poll or receive a webhook.

Testing

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_ask_rejects_short_question():
    r = client.post("/ask", json={"user_id": "u1", "question": "hi"})
    assert r.status_code == 422
```

No running server needed. Combine with `dependency_overrides` to fake the LLM for fast, free, deterministic tests.

Configuration

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env")
    openai_api_key: str = ""     # read from OPENAI_API_KEY

@lru_cache
def get_settings() -> Settings:
    return Settings()
```

12-Factor rule: config lives in the environment. Commit `.env.example`, never `.env`. Production secrets go in a secret manager.

6. Authentication, authorization and security

Authn vs authz CORE

- **Authentication**: who are you? Failure = 401.
- **Authorization**: what may you do? Failure = 403.

Mechanisms

Mechanism	How it works	Typical use
API key	Static secret in a header	Server-to-server (your OpenAI key)
Session cookie	Server stores session, browser sends cookie	Traditional web apps
JWT	Signed token with claims (`sub`, `exp`); verified without a DB lookup	Mobile and SPA backends, microservices
OAuth2	Delegated access: "app X may read my data at service Y"	Google Health, FatSecret, "Login with Google"

> A JWT is **signed, not encrypted**. Anyone can base64-decode and read it. Never put secrets in it. Keep expiry short.

OAuth2 Authorization Code flow (+ PKCE) CORE

1. Your app redirects the user to Google with `client_id`, requested `scope`s, `redirect_uri`, a `state` value (CSRF protection) and a PKCE challenge.
2. The user logs in at Google and consents.
3. Google redirects back to your `redirect_uri` with a short-lived `code`.
4. Your backend exchanges the code (plus client secret / PKCE verifier) for an **access token** (short-lived, ~1 hour) and a **refresh token** (long-lived).
5. Call the API with `Authorization: Bearer <access_token>`. When it expires, use the refresh token to get a new one.

Forgetting step 5's refresh logic is the top reason integrations "randomly break after an hour". **Client Credentials** flow is the machine-to-machine variant with no user. **Scopes** limit what a token may do: request the minimum.

BOLA: the #1 API vulnerability CORE

```
@app.get("/users/{user_id}/biometrics")
def get_bio(user_id: str, user: User = Depends(get_current_user)):
    if user.id != user_id:
        raise HTTPException(status_code=403)   # without this, any logged-in user
    ...                                        # can read anyone's health data
```

Broken Object Level Authorization tops the OWASP API Security Top 10. Being logged in is not the same as being allowed. Check ownership on every object access.

Security checklist

- HTTPS everywhere; validate all input (Pydantic); parameterized SQL (never string-format queries).
- Never log secrets, tokens or health data; least-privilege keys and scopes.
- Rate-limit per user/key; hash passwords with bcrypt or argon2 (never store plain text).
- Health data is sensitive personal data under GDPR (special category). Know: consent, data minimization, right to deletion, encryption at rest.
- For LLM apps, also prompt injection (chapter 12).

7. Consuming external APIs

As an FDE you will spend more time calling other people's APIs (customer systems, Google, FatSecret, OpenAI) than building your own. Robust clients assume failure.

The checklist for every external call CORE

1. **Timeouts** always. No timeout means one hung upstream can hang your service.
2. **Retry only retryable errors**: 429, 5xx, network errors. Never retry 400/401/403/404/422.
3. **Exponential backoff with jitter**: ~1s, 2s, 4s, 8s plus randomness, so thousands of clients do not retry in sync (thundering herd).
4. **Respect** `Retry-After` when given.
5. **Idempotency keys** for non-idempotent writes: `Idempotency-Key: <uuid>` lets the server drop duplicates.
6. **Circuit breaker**: after repeated failures, stop calling for a while and fail fast or degrade gracefully.

```
import random
import time

import httpx

RETRYABLE = {429, 500, 502, 503, 504}

def get_json_with_retry(url: str, headers: dict, max_attempts: int = 5) -> dict:
    for attempt in range(max_attempts):
        try:
            r = httpx.get(url, headers=headers, timeout=10.0)
            if r.status_code not in RETRYABLE:
                r.raise_for_status()          # 4xx: fail fast, no retry
                return r.json()
            retry_after = r.headers.get("Retry-After", "")
        except httpx.TransportError:          # DNS, connection reset, timeout
            retry_after = ""
        delay = float(retry_after) if retry_after.isdigit() else min(2**attempt, 30) + random.random()
        time.sleep(delay)
    raise RuntimeError(f"Gave up on {url} after {max_attempts} attempts")
```

In real code, the `tenacity` library does this declaratively. Be able to write it by hand in an interview.

Webhooks

The other service calls *you* when something happens ("new sleep data").

```
import hashlib
import hmac

expected = hmac.new(WEBHOOK_SECRET, raw_body, hashlib.sha256).hexdigest()
if not hmac.compare_digest(expected, request.headers["X-Signature"]):
    raise HTTPException(status_code=401)
```

- Verify the signature over the *raw* body, or anyone can send you fake events. `compare_digest` prevents timing attacks.
- Respond 2xx quickly; do heavy work in the background, or the sender times out and retries.
- Expect duplicates and out-of-order delivery: deduplicate on event ID.

Polling vs webhooks

Polling is simple but wasteful and laggy; webhooks are real-time but more complex. Robust integrations use webhooks plus a periodic reconciliation sync to catch missed events.

Reading unfamiliar API docs: the five questions

1. How do I authenticate (API key, OAuth2 which flow, scopes)?
2. What are the rate limits and how are they signaled?
3. How does pagination work?
4. What does an error look like?
5. What are the data units and time zones (HRV in ms? timestamps UTC or local?)

Data normalization matters: map every provider into *your* schema (an adapter per provider), so the rest of the system never depends on Google's or FatSecret's format.

8. Python essentials for backend work

Environment

- **Virtual environment** (`python -m venv .venv`): isolated packages per project.
- `requirements.txt` (or `pyproject.toml` with uv/poetry) pins dependencies. Lock exact versions for reproducible deploys.

Language features you will use daily

Feature	Example	Why it matters
Type hints	`def f(x: int) -> str`	FastAPI and Pydantic are driven by them; editors catch bugs
Dataclass vs Pydantic	`@dataclass` / `BaseModel`	Dataclass = plain container; Pydantic = validates and parses untrusted input
Decorators	`@app.get(...)`, `@lru_cache`	Functions that wrap or register other functions
Context managers	`with open(...)`, `with Session() as s`	Guaranteed cleanup of files, connections, transactions
Generators	`yield`	Lazy sequences; streaming responses; FastAPI dependencies with cleanup
async / await	`await client.get(url)`	Concurrency for I/O-bound work on one thread
Exceptions	`try / except / raise`	Catch specific exceptions, never a bare `except:`

async in one picture

```
import asyncio
import httpx

async def fetch_all(urls: list[str]) -> list[int]:
    async with httpx.AsyncClient(timeout=10) as client:
        responses = await asyncio.gather(*(client.get(u) for u in urls))
    return [r.status_code for r in responses]
```

Ten 1-second requests take about 1 second total instead of 10, because the event loop switches tasks while each one waits on the network. Async does not make CPU work faster.

Concurrency vocabulary

- **I/O-bound** (network, disk): use async or threads.
- **CPU-bound** (math, local models): use multiple processes; the GIL limits threads for CPU work in standard CPython.

9. SQL and relational databases

Core concepts CORE

- **Table** = rows (records) and columns (typed fields).
- **Primary key (PK)**: unique identifier of a row (`users.id`).
- **Foreign key (FK)**: column referencing another table's PK (`daily_biometrics.user_id -> users.id`); the DB enforces the relationship.
- **Constraints**: `NOT NULL`, `UNIQUE (user_id, recorded_on)` (one row per user per day; enables idempotent upserts), `CHECK (sleep_score BETWEEN 0 AND 100)`.
- **Index**: a data structure (usually a B-tree) that makes lookups fast, like a book index. Speeds reads, slows writes slightly. Index columns you filter or join on.

Our schema

```
CREATE TABLE users (
    id          UUID PRIMARY KEY,
    email       TEXT UNIQUE NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE daily_biometrics (
    id            UUID PRIMARY KEY,
    user_id       UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    recorded_on   DATE NOT NULL,
    hrv_ms        REAL,
    sleep_score   INT CHECK (sleep_score BETWEEN 0 AND 100),
    resting_hr    INT,
    calories_kcal INT,
    UNIQUE (user_id, recorded_on)
);
CREATE INDEX ix_bio_user_date ON daily_biometrics (user_id, recorded_on);
```

Queries you must be able to write CORE

```
-- Filter, sort, limit
SELECT recorded_on, hrv_ms FROM daily_biometrics
WHERE user_id = :uid ORDER BY recorded_on DESC LIMIT 7;

-- JOIN
SELECT u.email, b.recorded_on, b.hrv_ms
FROM users u
JOIN daily_biometrics b ON b.user_id = u.id
WHERE b.recorded_on >= CURRENT_DATE - INTERVAL '7 days';

-- Aggregate with GROUP BY / HAVING
SELECT user_id, AVG(sleep_score) AS avg_sleep
FROM daily_biometrics GROUP BY user_id HAVING AVG(sleep_score) < 60;

-- Window function: 7-day rolling HRV baseline
SELECT recorded_on, hrv_ms,
       AVG(hrv_ms) OVER (ORDER BY recorded_on
                         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS hrv_7d_avg
FROM daily_biometrics WHERE user_id = :uid;

-- Upsert (idempotent ingestion from Google Health)
INSERT INTO daily_biometrics (id, user_id, recorded_on, hrv_ms)
VALUES (:id, :uid, :day, :hrv)
ON CONFLICT (user_id, recorded_on) DO UPDATE SET hrv_ms = EXCLUDED.hrv_ms;
```

Know the JOIN types: `INNER` (only matches), `LEFT` (all left rows, NULLs when no match). `WHERE` filters rows before grouping; `HAVING` filters groups after.

Transactions and ACID

- **Atomicity**: all statements succeed or none do.
- **Consistency**: constraints always hold.
- **Isolation**: concurrent transactions don't see each other's half-done work.
- **Durability**: committed data survives crashes.

Design and performance

- **Normalization**: store each fact once (user email only in `users`). Denormalize deliberately for read speed.
- **N+1 problem**: 1 query for 100 users, then 100 queries for their biometrics. Fix with a JOIN or eager loading.

- **SQL injection**: never build SQL with f-strings from user input. Use parameters (`:uid`) or the ORM.
- `EXPLAIN ANALYZE` shows how Postgres executes a query (index scan vs sequential scan).
- **Connection pooling**: reuse DB connections instead of opening one per request.

ORM and migrations

```
class DailyBiometrics(Base):
    __tablename__ = "daily_biometrics"
    id: Mapped[UUID] = mapped_column(primary_key=True, default=uuid4)
    user_id: Mapped[UUID] = mapped_column(ForeignKey("users.id"), index=True)
    hrv_ms: Mapped[float | None]

stmt = select(DailyBiometrics).where(DailyBiometrics.user_id == uid).limit(7)
rows = session.scalars(stmt).all()
```

SQLAlchemy maps Python classes to tables. **Alembic** manages *migrations*: versioned scripts that evolve the schema (add a column) safely in every environment. Never edit production schemas by hand.

SQL vs NoSQL vs vector DB

Type	Strength	In our app
Relational (Postgres)	Structure, joins, transactions, exact queries	Users, biometrics, meals, tokens
Document / key-value (MongoDB, Redis)	Flexible schemas, very fast simple lookups, caching	Redis for caching and rate limits
Vector (Qdrant, pgvector)	Similarity search over embeddings	Protocol knowledge base

10. Embeddings and vector databases

What an embedding is CORE

An embedding model turns text into a list of numbers (a vector) such that **texts with similar meaning end up close together** in that space. `text-embedding-3-small` produces 1536 numbers per text. "CNS fatigue after heavy squats" and "nervous system feels fried" land near each other even with no shared words. That is the advantage over keyword search.

```
from openai import OpenAI

client = OpenAI()
resp = client.embeddings.create(model="text-embedding-3-small",
                                input=["HRV dropped 15% today", "Zone 2 cardio for recovery"])
vectors = [d.embedding for d in resp.data]     # two lists of 1536 floats
```

Rule: the query and the documents must be embedded with the **same model**. Changing models means re-embedding everything.

Similarity metrics CORE

Cosine similarity = cos(angle between vectors) = (a · b) / (|a| |b|). Range -1 to 1; higher = more similar. It ignores vector length and compares direction only.

```
import numpy as np

def cosine(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))
```

- **Dot product**: equals cosine when vectors are normalized to length 1 (OpenAI embeddings are), and is cheaper to compute.
- **Euclidean distance**: straight-line distance; lower = more similar.

Why a vector database? CORE

Brute force compares the query with every vector: cost grows with N x dimensions. Fine for 1,000 documents, too slow for 10 million. Vector DBs use **Approximate Nearest Neighbor (ANN)** indexes, which trade a little accuracy for huge speed.

- **HNSW** (Hierarchical Navigable Small World): a layered graph; search starts at a sparse top layer and greedily descends to dense lower layers. Roughly logarithmic search time. Parameters (`m`, `ef`) trade recall vs speed and memory. Used by Qdrant, pgvector, Weaviate.
- **IVF**: cluster vectors and search only nearby clusters. **PQ** (product quantization): compress vectors to save memory.
- Beyond the index, a vector DB also gives you persistence, metadata (payload) filtering, updates and deletes, replication and an API.

Choosing a vector store

Option	Pros	Cons
Qdrant (our choice)	Open source, runs embedded in Python or as a server, strong payload filtering, fast (Rust)	One more system to operate in production
Pinecone	Fully managed, zero ops	Paid, cloud-only, vendor lock-in, cannot run locally
pgvector	Lives inside Postgres: one database, SQL joins with user data, transactions	Less specialized at very large scale
Chroma	Very easy for prototypes	Less proven in production
FAISS	Extremely fast library from Meta	A library, not a database: no persistence, filtering or API out of the box

Qdrant basics

```
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, FieldCondition, Filter, MatchValue, PointStruct, VectorParams

client = QdrantClient(path="./qdrant_data")           # embedded mode, no Docker
client.create_collection("protocols",
    vectors_config=VectorParams(size=1536, distance=Distance.COSINE))

client.upsert("protocols", points=[
    PointStruct(id=1, vector=vec, payload={"title": "HRV autoregulation", "category": "recovery"}),
])

hits = client.query_points("protocols", query=query_vec, limit=3,
    query_filter=Filter(must=[FieldCondition(key="category", match=MatchValue(value="recovery"))]),
).points
for h in hits:
    print(h.score, h.payload["title"])
```

Vocabulary: **collection** (like a table), **point** (id + vector + payload), **payload** (metadata JSON you can filter on).

11. The RAG pipeline and evaluation

Two pipelines CORE

Ingestion (offline): load documents, clean them, split into chunks, embed each chunk, upsert into the vector DB with metadata (source, title, category, date).

Query (online): embed the question, retrieve top-k chunks (with filters), optionally rerank, build a prompt (instructions + user data + chunks + question), generate, return the answer with cited sources.

Chunking

- Why: embedding a whole 30-page document blurs its meaning into one vector, and you cannot fit everything in the prompt.
- Strategies: fixed size (e.g. ~300-800 tokens) with overlap (10-20%) so ideas are not cut in half; recursive splitting on paragraphs and sentences; semantic or structure-aware splitting (headings, sections).
- Trade-off: small chunks are precise but lose context; large chunks keep context but dilute relevance and cost more tokens.
- Store parent document ID in metadata so you can fetch neighbors or cite the source.

Improving retrieval

Technique	What it does
Metadata filtering	Restrict search (category = nutrition, language = tr, user's injuries)
Hybrid search	Combine dense vectors (meaning) with BM25 keyword search (exact terms like "RPE", drug names). Merge with Reciprocal Rank Fusion
Reranking	Retrieve ~20-50 candidates fast, then rescore with a cross-encoder that reads query and chunk together; keep the best 3-5. More accurate, slower
Query rewriting	LLM turns a vague or conversational question into a better search query (or several)
Contextual retrieval	Enrich the query with structured context ("HRV -15% vs baseline") before embedding

Failure modes CORE

- **Retrieval miss**: the right chunk was not retrieved, so the LLM answers from its own memory (hallucination risk).
- **Irrelevant context**: wrong chunks distract or mislead the model.
- **Lost in the middle**: models use information at the start and end of long contexts better than the middle. More chunks is not always better.
- **Stale index**: documents changed but embeddings were not updated.
- **Unfaithful answer**: correct chunks retrieved, but the answer contradicts or goes beyond them.

Mitigations: instruct the model to answer only from context and say "I don't know" otherwise, require citations by chunk ID, set a similarity score threshold, and evaluate.

Evaluation: "how do you know it works?" CORE

Build a **golden set**: 30-100 realistic questions, each labeled with the chunk(s) that should be retrieved and key facts the answer must contain. Evaluate the two halves separately.

Stage	Metric	Meaning
Retrieval	Recall@k (hit rate)	Share of questions where a correct chunk appears in the top k
Retrieval	MRR	Mean of 1 / rank of the first correct chunk (rewards ranking it first)
Retrieval	Precision@k	Share of the top k that is relevant
Generation	Faithfulness / groundedness	Is every claim supported by the retrieved context?
Generation	Answer relevance / correctness	Does it answer the question; does it contain the key facts?

LLM-as-judge: a strong model grades answers against a rubric. Cheap and scalable, but validate it against human labels on a sample. Tools: Ragas, DeepEval, promptfoo. Run the eval on every change (chunk size, model, prompt) so improvements are measured, not guessed.

12. LLMs and LLM APIs

Core vocabulary CORE

- **Token**: the unit models read and bill by; roughly 4 characters or 0.75 English words. Turkish usually needs more tokens per word.
- **Context window**: max tokens of input + output per call. Bigger is not free: cost, latency and "lost in the middle".
- **Temperature**: randomness. Low (0-0.3) for factual, consistent answers; higher for creative text.
- **Roles**: `system` (rules and persona), `user` (input), `assistant` (model output). The API is stateless: you resend the conversation history every call.
- **Hallucination**: fluent but false output. RAG, citations and evaluation reduce it; nothing removes it fully.

A chat completion

```
resp = client.chat.completions.create(
    model="gpt-4o-mini",
    temperature=0.2,
    messages=[
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": f"Biometrics:\n{bio}\n\nProtocols:\n{chunks}\n\nQuestion: {q}"},
    ],
)
answer = resp.choices[0].message.content
usage = resp.usage           # prompt_tokens, completion_tokens: log these for cost tracking
```

Prompt structure for RAG

1. Role and goal ("You are an evidence-informed strength and recovery coach").
2. Rules: use only the provided context; cite protocol IDs; say when information is missing; no medical diagnosis; recommend a professional for red flags.
3. Clearly delimited data sections (user biometrics, retrieved protocols with IDs).
4. The question and the desired output format.

Structured outputs and tool calling

- **Structured outputs**: force the model to return JSON matching a schema (e.g. a Pydantic `WorkoutPlan`), then validate it. Turns text into data your backend can use.
- **Tool / function calling**: you describe functions (`get_biometrics(days)`, `search_protocols(query)`); the model replies with which function to call and arguments; *your code* executes it and sends the result back. This loop is the basis of "agents". The model never executes anything itself.

Cost, latency and reliability

- Cost = input tokens x price + output tokens x price. Output tokens are typically several times more expensive. Fewer, better chunks save money.
- Latency: use smaller models where possible, stream output, cache repeated work, run independent calls in parallel.
- Provider errors happen (429, 5xx, timeouts): apply chapter 7. Consider a fallback model.

Prompt injection and guardrails CORE

Untrusted text (user input, retrieved documents, web pages, API data) can contain instructions like "ignore previous instructions". Mitigations:

- Treat retrieved content as data: delimit it and tell the model it is reference material, not instructions.
- Least privilege for tools: the model should not be able to delete data or access other users' records.
- Validate outputs (schemas, allow-lists), and enforce authorization in code, never in the prompt.
- Domain guardrails: detect medical red flags (chest pain, fainting) and respond with a safe, fixed message.

When to use what

Need	Approach
Better format or instructions	Prompt engineering, few-shot examples
Knowledge that changes or is private	RAG
Consistent style or narrow task at lower cost	Fine-tuning (often a small model)
Taking actions, multi-step work	Tool calling / agents

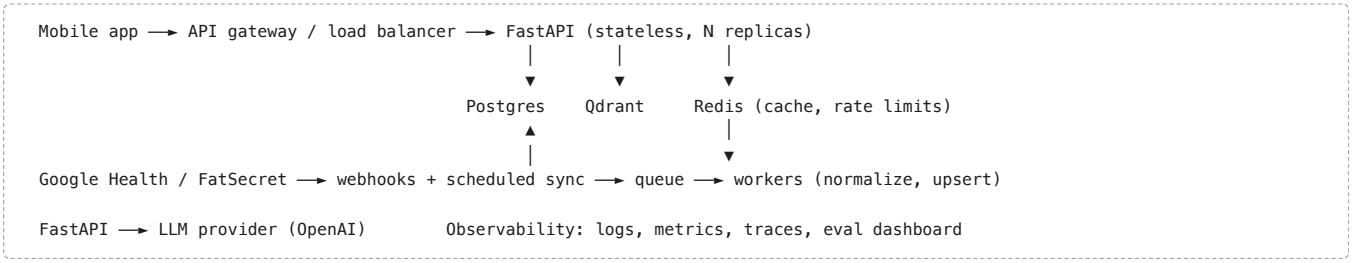
13. System design for AI applications

Interview framework CORE

1. **Clarify requirements**: functional (what it does) and non-functional (latency, scale, availability, privacy, cost).
2. **Estimate**: users, requests per second, data size, token cost per request.
3. **API**: the main endpoints and their inputs and outputs.
4. **Data model**: what lives in SQL, vector DB, cache, object storage.
5. **High-level architecture**: draw boxes and arrows.
6. **Deep dive**: the hardest part (retrieval quality, ingestion at scale, sync with external APIs).
7. **Trade-offs and failure modes**: what breaks first, how you monitor and evolve it.

Always say the trade-off out loud: "I chose X over Y because..., at the cost of...".

Reference architecture for our coach



Key building blocks

- **Stateless API + load balancer**: scale horizontally by adding replicas.
- **Queue + workers** (Redis/RQ, Celery, SQS): decouple slow or bursty work (ingestion, nightly plan generation) from user requests; retries and backpressure.
- **Caching**: embedding cache (same text, same vector), response cache for repeated questions, cache-aside pattern; always define invalidation.
- **Rate limiting**: token bucket per user; protects you from abuse and runaway LLM bills.
- **Observability**: structured logs with request IDs; metrics (p50/p95/p99 latency, error rate, tokens and cost per request); tracing (OpenTelemetry, Langfuse) showing time per step: "2.1s in LLM, 40ms in Qdrant".
- **Reliability**: timeouts, retries, circuit breakers, graceful degradation (if the LLM is down, return a rule-based recommendation).
- **Privacy**: encrypt at rest and in transit, minimize data sent to the LLM, per-user isolation, audit logs, deletion on request.

Useful numbers (orders of magnitude)

Operation	Typical latency
Redis lookup	< 1 ms
Indexed Postgres query	1-10 ms
Vector search (ANN, ~1M vectors)	5-50 ms
Embedding API call	50-300 ms
LLM time to first token	200 ms - 1 s
Full LLM answer (300 tokens)	2-10 s

Takeaway: in an AI app, the LLM dominates latency and cost. Optimize there first.

14. What makes a great Forward Deployed Engineer

An FDE sits between the product and the customer: you take a general AI platform and make it solve one customer's real problem, often inside their messy environment.

Skills interviewers look for

- **Problem discovery**: ask what success looks like, who the users are, what data exists, and what constraints apply (security, compliance, latency) before building.
- **Scoping**: cut a big ask into a demo-able first version in days, not months.
- **Integration under uncertainty**: unfamiliar APIs, missing docs, odd data formats; chapter 7 is your daily life.
- **Debugging in production**: logs, reproducing with `curl`, isolating which layer fails.
- **Communication**: explain trade-offs to non-engineers, write clear summaries, say "I don't know yet, here's how I'll find out".
- **Ownership**: you are the person who makes it work end to end.

Typical interview formats

- Live integration: "Here are the docs for API X; build a small service that pulls data and answers questions about it."
- Decomposition / case: "A hospital wants an assistant over its internal guidelines. Walk me through it."
- Debugging: a broken repo or failing request to fix while thinking aloud.
- Project deep dive: your fitness coach. Be ready for "why Qdrant?", "how did you evaluate retrieval?", "what would break at 100x users?".

> **Your project pitch (practice it):** "I built a RAG-based fitness coach. FastAPI backend; user biometrics in Postgres, synced from Google Health via OAuth2 with idempotent upserts; recovery protocols embedded in Qdrant. Each request combines exact user data with semantically retrieved protocols and GPT generates a cited plan. I built a golden eval set and improved recall@5 from X to Y with hybrid search and reranking."

15. Interview question bank

Answer out loud before reading the answer. If you can't, mark it for review.

HTTP and REST

PUT vs PATCH?

> PUT replaces the whole resource and is idempotent; PATCH updates part of it.

401 vs 403?

> 401: not authenticated. 403: authenticated but not permitted.

What is idempotency and why does it matter?

> Repeating a request has the same effect as doing it once. It makes retries safe on unreliable networks. For POST, use idempotency keys.

Which errors should a client retry?

> 429, 5xx and network errors, with exponential backoff and jitter, respecting Retry-After. Never 4xx validation or auth errors.

Offset vs cursor pagination?

> Offset is simple but slow and unstable on large or changing data; cursor is stable and fast.

What does "stateless" mean and why care?

> Each request contains everything needed; servers keep no client session state, so you can scale horizontally behind a load balancer.

How do you version an API?

> URL prefix (/v1) or header; additive changes stay in the same version, breaking changes get a new version with a deprecation period.

FastAPI and Python

Why FastAPI?

> Pydantic validation, auto OpenAPI docs, async support, performance; standard for ML serving.

What happens when the request body is invalid?

> Pydantic validation fails and FastAPI returns 422 with details before the handler runs.

What is dependency injection in FastAPI and why use it?

> Depends() runs provider functions (DB session, current user, clients) and injects results. Removes duplication and allows overriding dependencies in tests.

def vs async def?

> def runs in a thread pool; async def runs on the event loop and must not block. Blocking inside async def stalls all requests.

How do you test an endpoint that calls OpenAI?

> TestClient plus dependency_overrides to replace the LLM client with a fake returning fixed output.

How do you serve a slow LLM response well?

> Stream tokens (SSE / StreamingResponse) for chat; for batch work return 202 with a job ID and process in a worker.

Security

Explain the OAuth2 Authorization Code flow.

> Redirect to provider with scopes and state, user consents, provider redirects back with a code, backend exchanges code for access + refresh tokens, refresh when the access token expires. PKCE protects the code exchange.

What is BOLA?

> Broken Object Level Authorization: users can access other users' objects by changing an ID. Prevent by checking ownership on every access.

Is a JWT encrypted?

> No, it is signed. Anyone can read its payload; the signature only proves it wasn't tampered with.

How do you verify a webhook?

> Compute an HMAC of the raw body with the shared secret and compare with compare_digest; respond fast and deduplicate by event ID.

SQL

What is an index and what does it cost?

> A structure (usually B-tree) for fast lookups on a column; speeds reads, costs storage and slightly slower writes.

WHERE vs HAVING?

> WHERE filters rows before grouping; HAVING filters groups after aggregation.

What is the N+1 problem?

> One query for a list, then one query per item. Fix with JOINs or eager loading.

Explain ACID.

> Atomicity, Consistency, Isolation, Durability: transactions are all-or-nothing, keep constraints valid, don't interfere, and persist after commit.

How do you make ingestion idempotent?

> A unique constraint on the natural key (user_id, date) plus INSERT ... ON CONFLICT DO UPDATE.

Vectors and RAG

Training vs RAG?

> Training changes weights (behavior, style); RAG injects retrieved context at request time (facts, fresh and private data, citations). RAG is cheaper to update and auditable.

What is an embedding?

> A vector representation of text where semantic similarity becomes geometric closeness.

Cosine similarity vs dot product?

> Cosine compares direction only; for normalized vectors they are identical and dot product is cheaper.

Why not just brute-force search?

> Cost grows linearly with N; ANN indexes like HNSW give near-logarithmic search with a small recall trade-off.

Qdrant vs Pinecone vs pgvector?

> Qdrant: open source, local or server, strong filtering. Pinecone: managed, lock-in, paid. pgvector: one DB with SQL joins, less specialized at large scale.

How do you choose chunk size?

> Start with a few hundred tokens with overlap, then measure recall on a golden set; small = precise but less context, large = more context but diluted.

How do you evaluate a RAG system?

> Golden set; retrieval metrics (recall@k, MRR) and generation metrics (faithfulness, relevance) via human or LLM-as-judge; run on every change.

Retrieval returns nothing relevant. What should happen?

> Score threshold; the model says it lacks information instead of guessing; log the query to improve the knowledge base.

What is hybrid search?

> Combining dense vector search with keyword search (BM25) and fusing results; helps with exact terms and rare words.

What is reranking?

> Retrieve many candidates cheaply, then rescore with a cross-encoder that reads query and chunk together; better precision at extra latency.

LLMs and system design

What is prompt injection and how do you mitigate it?

> Untrusted text containing instructions. Delimit data, least-privilege tools, validate outputs, enforce authorization in code.

How does tool calling work?

> The model outputs a function name and arguments; your code executes it and returns the result to the model, looping until done.

How would you reduce LLM cost?

> Smaller models for easy tasks, fewer/better chunks, caching, shorter prompts, batching, limiting output length, tracking tokens per request.

What breaks first at 100x users?

> LLM rate limits and cost, then DB connections and sync jobs. Answer with: queues, caching, rate limits, replicas, connection pooling, monitoring.

How would you design ingestion from Google Health for 1M users?

> OAuth tokens stored encrypted; webhooks plus scheduled sync; a queue with workers; per-user rate limiting and backoff; idempotent upserts; normalization adapter; monitoring of lag and failures.

16. Checklists and debugging toolkit

Debugging an API problem

1. What is the status code?
2. What does the response body say?
3. Are the headers right (auth, content type)?
4. Can I reproduce it with `curl`?
5. What do the server logs say for this request ID?
6. Which layer fails: network, auth, validation, business logic, database, upstream API?

```
curl -i localhost:8000/health                     # -i: show status and headers
curl -v https://api.example.com/x                 # -v: full exchange incl. TLS
curl -X POST localhost:8000/ask \
  -H 'Content-Type: application/json' \
  -d '{"user_id":"u1","question":"How should I deload?"}'
```

Also: Postman or Bruno (GUI clients), the browser DevTools Network tab (CORS issues), FastAPI's `/docs`.

Production-readiness checklist for an endpoint

- ☐ Input validated with Pydantic; output via `response_model`
- ☐ Authentication and object-level authorization
- ☐ Correct status codes and a consistent error format
- ☐ Timeouts and retries on every external call
- ☐ Structured logging with request ID; no secrets or health data in logs
- ☐ Rate limiting
- ☐ Tests for the happy path and main failure cases
- ☐ Documented in OpenAPI with examples

Phase 1 progress

- ☐ Step 1: archive scaffold, first commit (done)
- ☐ Step 2: FastAPI app with /health and Pydantic models (done)
- ☐ Step 3: mock data (protocols.json, users.json)
- ☐ Step 4: embeddings + NumPy cosine similarity
- ☐ Step 5: Qdrant ingestion and filtered search
- ☐ Step 6: prompt assembly and generation
- ☐ Step 7: POST /ask end to end with sources and scores

Recommended resources

- FastAPI official tutorial (fastapi.tiangolo.com): read "Tutorial - User Guide" fully.
- MDN Web Docs: HTTP overview, status codes, CORS.
- OWASP API Security Top 10.
- SQLBolt or PostgreSQL tutorial for hands-on SQL; "Use The Index, Luke" for indexes.
- Qdrant documentation: concepts (collections, points, payload, filtering, HNSW).
- OpenAI API docs: embeddings, structured outputs, function calling.
- Chip Huyen, "AI Engineering" (book) and "Designing Machine Learning Systems".
- Alex Xu, "System Design Interview" for the general framework.